# HW3_Team1

**Exercise 48:**

Consider a floating-point number system with base $b$, precision $p$, and exponent range $[e_{\min}, e_{\max}]$.

(a) How many different non-zero normalized numbers are there?

Normalized numbers have the following general form:

$$x = \sigma(\delta_0 + \frac{\delta_1}{b^1} + \cdots + \frac{\delta_{p-1}}{b^{\,p-1}})b^{\,e}$$

where $e \in [e_{min}, e_{max}],\ i \in \{0, \dots, p-1\},\ \delta_i \in \{0, \dots, b-1\}$ and $\delta_0 \neq 0$.

For $\delta_0$ we choose from $b-1$ numbers (excl. 0), while for each of the remaining $p-1$ digits $\delta_1, \dots, \delta_{p-1}$ we choose from $b$ numbers. For the exponent we choose from $e_{max} - e_{min} + 1$ integers and for the sign we have 2 possibilities. Hence, the total amount of numbers is obtained as

$$2 \cdot (b-1) \cdot b^{\,p-1} \cdot (e_{max} - e_{min} + 1)$$

Since we need normalized numbers, we have $\delta_0 \neq 0$ and hence all 0s are excluded. Moreover, zero cannot be normalized.

(b) What are the smallest (normalized) and largest positive numbers?

The smallest normalized positive number is:

$$x_s = +(1 + \frac{0}{b^1} + \cdots + \frac{0}{b^{\,p-1}})b^{e_{min}}$$

The largest normalized positive number is.

$$x_l = +((b-1) + \frac{b-1}{b^1} + \cdots + \frac{b-1}{b^{\,p-1}})b^{e_{max}} = b(1 - b^{-p})b^{e_{max}}$$

**Exercise 49:**

Consider a floating-point number system with base b, precision p, and exponent range $[e_{\min}, e_{\max}]$.

If $b = 10$, what are the smallest values of $p$ and $e_{\min}$ and largest value of $e_{\max}$such that both 2365.27 and 0.0000512 can be represented exactly as normalized floating-point numbers?

To answer this, we first need to normalize both values:

2365.27 normalized corresponds to: $2.36527 * 10^3$

0.0000512 normalized corresponds to: $5.12 * 10^{-5}$

Deductions:

The significand of the first number is $6_{digits}$ long, meaning we need a precision $p$ of at least 6. Furthermore we need the upper bound of the exponent range to be greater than or equal to 3.

We need: $e_{max} \geq 3$

The significant of the second number is 3 digits long, which is already covered by the precision of 6. Furthermore, we need the lower bound of the exponent range to be $\leq -5$.

We need: $e_{min} \leq -5$

Therefore the exact tightest Bounds are:

$$p = 6, e_{min} = -5, e_{max} = 3$$

**Exercise 50:**

In a floating-point number system with precision $p = 6$ decimal digits, let $x = 1.23456$ and $y = 1.23579$.

(a) How many significant digits does the difference $y - x$ contain?

(b) What is the minimum exponent range for which $x$, $y$ and $y-x$ are all exactly representable as normalized numbers?

A normalized decimal number with $p = 6$ digits has the form $d_0.d_1d_2d_3d_4d_5 \cdot 10^e$ with $d_0 \neq 0$. Subtracting gives

$$y - x = 1.23579 - 1.23456 = 0.00123 = 1.23000 \cdot 10^{-3}.$$

The leading digits cancel, so only the digits 123 are left.

```r
x <- 1.23456
y <- 1.23579

# R uses binary numbers, so the result is not exactly 0.00123
print(y - x)
```

```
[1] 0.00123
```

```r
print(y - x, digits = 16)
```

```
[1] 0.001229999999999842
```

```r
# exponents of x, y and y - x: 0, 0 and -3
format(c(x, y, y - x), scientific = TRUE)
```

```
[1] "1.23456e+00" "1.23579e+00" "1.23000e-03"
```

(a) The difference has only 3 significant digits. The other 3 digits of the 6-digit result are just zeros that carry no information.

(b) $x$ and $y$ have exponent 0 and $y - x = 1.23000 \cdot 10^{-3}$ has exponent $-3$. So the minimum exponent range is $[e_{min}, e_{max}] = [-3, 0]$.

**Exercise 51:**

In single precision four bytes are used to represent a floating point number: 1 bit is used for the sign, 8 for the exponent, and 23 for the fraction. What are the largest and smallest non-zero positive numbers in single precision (including denormalized numbers)?

A number is stored as $\pm\text{mantissa} \cdot 2^{\text{exponent}}$. For the largest number we take the biggest mantissa and the biggest exponent, for the smallest number the smallest ones.

Mantissa example in the decimal system: $5230000 = 5.23 \cdot 10^6$ has mantissa 5.23 and exponent 6

Exponent : 8 bits give the values 0 to 255 and we subtract the bias 127. Two values are special: 255 is used for $\infty$ and NaN/0 is used for denormalized numbers. So normal numbers have exponents from $1 - 127 = -126$ to $254 - 127 = 127$.

For the largest number the biggest exponent is 127. The biggest mantissa has all 23 fraction bits equal to 1:

$$1.11\ldots1 = 1 + \tfrac{1}{2} + \cdots + \tfrac{1}{2^{23}} = 2 - 2^{-23}$$

which is almost 2. So the largest number is $(2 - 2^{-23}) \cdot 2^{127}$

For the smallest number denormalized numbers we start with 0 instead of 1 (like $0.xxx\ldots$) and use the exponent $-126$. The smallest one has only the last bit equal to 1, so its mantissa is $0.00\ldots01 = 2^{-23}$. So the smallest number is $2^{-23} \cdot 2^{-126} = 2^{-149}$.

```
n_bits <- 23 #number of fraction bits
e_max <- 127  #largest exponent
e_min <- -126 #smallest exponent

largest <- (2 - 2^(-n_bits)) * 2^e_max
#biggest mantissa (all fraction bits are basically 1) * biggest exponent
largest
```

```
[1] 3.402823e+38
```

```
smallest <- 2^(-n_bits) * 2^e_min
#denormalized, so only the last fraction bit is 1
smallest
```

```
[1] 1.401298e-45
```

So the largest positive number is about $3.4028 \cdot 10^{38}$ and the smallest positive number is about $1.4013 \cdot 10^{-45}$

**Exercise 52:**

A normalized number has the form

$$x = \pm\,(d_0.d_1d_2\ldots d_{p-1})_b \cdot b^e, \qquad d_0 \neq 0, \quad e_{\min} \leq e \leq e_{\max}.$$

Two adjacent numbers with the same exponent $e$ differ by one unit in the last digit $d_{p-1}$, which is worth $b^{-(p-1)}$. Scaled by $b^e$, the spacing is

$$y - x = b^{-(p-1)} \cdot b^e = b^{e-p+1}.$$

So the spacing only depends on the exponent. It is smallest for the smallest exponent and largest for the largest exponent:

$$\text{min spacing} = b^{e_{\min}-p+1}, \qquad \text{max spacing} = b^{e_{\max}-p+1}.$$

```r
spacing <- function(b, p, e_min, e_max) {
  min_spacing <- b^(e_min - p + 1)
  max_spacing <- b^(e_max - p + 1)
  c(min = min_spacing, max = max_spacing)
}

# small example: b = 10, p = 3, exponents from -2 to 2
spacing(10, 3, -2, 2)
```

```
  min   max
1e-04 1e+00
```

```r
# IEEE double precision: b = 2, p = 53, exponents from -1022 to 1023
spacing(2, 53, -1022, 1023)
```

```
          min           max
4.940656e-324 1.995840e+292
```

For double precision this gives $2^{-1074}$ and $2^{971}$. We can check both values in R: the spacing next to the smallest normalized number $2^{-1022}$, and the spacing just below the largest number `.Machine$double.xmax`.

```r
 x_small <- 2^-1022 # smallest normalized number
(x_small + 2^-1074) - x_small == 2^-1074 # the next number up is exactly 2^-1074 away
```

```
[1] TRUE
```

```r
x_big <- .Machine$double.xmax # largest number, all digits 1 with e = 1023
x_big - (x_big - 2^971) == 2^971 # the number below it is exactly 2^971 away
```

```
[1] TRUE
```

**Exercise 53:**

In many floating-point systems, a quick approximation to the unit roundoff error can be obtained by the approximation

$$\epsilon_{\text{mach}} \approx |3 \cdot (4/3 - 1) - 1|.$$

What is the idea behind this "trick"? Does it work when using IEEE 754 double precision?

In regular arithmetic this is 0, but in floating-point arithmetic it is not. Since $4/3$ is an infinite (non-terminating) decimal ($1.333\ldots$), it is rounded to the nearest floating-point number in the given system.

Let $\text{float}(x)$ represent the floating-point variant of $x$. Then the floating-point representation of $\text{float}(\frac{4}{3}) = \frac{4}{3} + \eta$ is given by $\frac{4}{3}$ and some rounding error $\eta$. Then we have

$$\begin{aligned}
\epsilon_{\text{mach}} &\approx \left|3(\text{float}\left(\frac{4}{3}\right) - 1) - 1\right| = \\
&= \left|3\left(\frac{4}{3} + \eta - 1\right) - 1\right| = \\
&= \left|3\left(\frac{1}{3} + \eta\right) - 1\right| = \\
&= |1 + 3\eta - 1| = \\
&= |3\eta|
\end{aligned}$$

This trick also works with IEEE 754 double precision. Consider $\frac{4}{3}$ in binary $\frac{4}{3} = (1.010101\ldots)_2$. In IEEE 754 we have 52 bits for the significant, hence the significant is $0101\ldots$ (the first bit is skipped). What gets discarded then is $(010101\ldots)_2 \times 2^{-52} = \frac{1}{3} \cdot 2^{-52}$. Hence, the floating-point representation of $\frac{4}{3}$ under IEEE 754 is

$$\text{float}\left(\frac{4}{3}\right) = \frac{4}{3} - \frac{1}{3} \cdot 2^{-52}$$

Similarly to the general example above, we have

$$\begin{aligned}
\epsilon_{mach} &\approx \left|3 * \left(\text{float}\left(\frac{4}{3}\right) - 1\right) - 1\right| = \\
&= \left|3 * \left(\frac{4}{3} - \frac{1}{3} \cdot 2^{-52} - 1\right) - 1\right| = \\
&= \left|3 * \left(\frac{1}{3} - \frac{1}{3} \cdot 2^{-52}\right) - 1\right| = \\
&= \left|1 - 2^{-52} - 1\right| = \\
&= 2^{-52}
\end{aligned}$$

**Exercise 54:**

```
x <- c(0, 7, 8)
x[0.9999999999999999]
```

```
numeric(0)
```

```
x[0.99999999999999999]
```

```
[1] 0
```

First, the vector x has 3 elements. If R receives an index that is not an integer, it does not round, but truncates the number after the decimal point (so 0.5 becomes 0, and 1.5 becomes 1).

Since the first number has 14 decimal digits, it falls well within R's double precision limit (~15.9 digits), so it is accurately represented as a fraction less than 1. Because it is not a whole number, R truncates it rather than rounding. Therefore, R evaluates the index as x[0].

R starts counting elements from 1 (so x[1] = 0), but the truncation leads to x[0].
In R, the index [0] does not address an element, but rather returns the empty shell or data type of the object.

So R returns a vector of length 0 with the corresponding data type (numeric).
Furthermore, if you were to specify a character vector like x_char <- c("A", "B", "C"), calling x_char[0] would return character(0), and for a logical vector, it would return logical(0).

Now for the second number: The distance between 0.9999999999999999 and 1 is now smaller than the machine epsilon. Because this exceeds the 53-bit precision limit, R cannot distinguish between these two numbers, so the R-parser reads and evaluates it exactly as 1.

Instead of truncating to 0, R now calls the element at the first position: x[1].
Since the first element in our vector is 0, R returns 0.

Side note:
By running .Machine$double.eps you can see the machine epsilon, roughly 2.2 * e^-16which is the smallest re-presentable difference between 1 and the next larger floating-point number in R.

### Exerice 55:

Write functions which use a starting value of 1 and repeated division by 2 to find the smallest numbers $\epsilon$ such that $1 + \epsilon \neq 1$ and $1 - \epsilon \neq 1$, respectively. What are these numbers?

We start with $\epsilon = 1$ (where $1 \pm \epsilon \neq 1$ holds) and keep halving as long as $1 \pm \epsilon/2$ is still different from 1. When the loop stops, halving one more time would make the sum equal to 1, so the current $\epsilon$ is the smallest one that still works.

```r
# smallest epsilon with 1 + epsilon != 1
epsilon_plus <- function() {
  epsilon <- 1
  while (1 + epsilon / 2 != 1) {
    epsilon <- epsilon / 2
  }
  return(epsilon)
}

# smallest epsilon with 1 - epsilon != 1
epsilon_minus <- function() {
  epsilon <- 1
  while (1 - epsilon / 2 != 1) {
    epsilon <- epsilon / 2
  }
  return(epsilon)
}

epsilon_plus()
```

```
[1] 2.220446e-16
```

```r
epsilon_minus()
```

```
[1] 1.110223e-16
```

```r
# comparison with 2^(-52), 2^(-53) and R's built-in values
epsilon_plus() == 2^(-52)
```

```
[1] TRUE
```

```r
epsilon_plus() == .Machine$double.eps
```

```
[1] TRUE
```

```r
epsilon_minus() == 2^(-53)
```

```
[1] TRUE
```

```
epsilon_minus() == .Machine$double.neg.eps
```

```
[1] TRUE
```

```
# why the two epsilons differ
1 + 2^(-53) == 1
```

```
[1] TRUE
```

```
1 + 2^(-52) == 1
```

```
[1] FALSE
```

```
1 - 2^(-54) == 1
```

```
[1] TRUE
```

```
1 - 2^(-53) == 1
```

```
[1] FALSE
```

The numbers are $\epsilon_+ = 2^{-52} \approx 2.22 \cdot 10^{-16}$ for $1 + \epsilon \neq 1$ and $\epsilon_- = 2^{-53} \approx 1.11 \cdot 10^{-16}$ for $1 - \epsilon \neq 1$. They match `.Machine$double.eps` and `.Machine$double.neg.eps`.

They differ by a factor of 2 because the spacing of the doubles is different above and below 1. Above 1 the spacing is $2^{-52}$, so $1 + 2^{-53}$ is exactly halfway between 1 and $1 + 2^{-52}$ and is rounded to 1. Below 1 the spacing is $2^{-53}$, so $1 - 2^{-53}$ is still representable, while $1 - 2^{-54}$ is rounded to 1. Additionally, as we halve each time, we only find powers of 2.

### Exercise 62:

Suppose $x$ and $y$ can be represented without error in double precision. Can the same be said for $x^2$ and $y^2$? Which would be more accurate, $x^2 - y^2$ or $(x + y)(x - y)$? For what values of $x$ and $y$, relative to each other, is there a substantial difference in the accuracy of the two expressions?

If $x$ and $y$ are represented without error in double precision, we can't be sure that $x^2$ or $y^2$ will also be represented without error. Suppose we have $x = 2^{52} + 1$. The double representation of this number will need 53 bits and its thus exactly re-presentable ($\delta_0 = 1, \delta_i = 0$ for $i \in$

$[1, 51], \delta_{52} = 1)$. Then $x^2 = (2^{52} + 1)^2 = 2^{104} + 2^{53} + 1$. This number will require 105 bits to be represented exactly. Thus, the $+1$ will be rounded away.

The difference of squares is therefore more accurately obtained using $(x + y)(x - y)$. If we use $x^2 - y^2$, then the squares are rounded first with small relative errors introduced (up to $2^{-53}$). The subtraction of the squares cancels the leading digits and the relative error will dominate the result. This is also known as "catastrophic cancellation". If we use $(x+y)(x-y)$ instead, then the subtraction is applied using exact numbers (by assumption). The formula $(x + y)(x - y)$ also avoids overflow and underflow of $x^2$ and $y^2$ for very large or very small values of $x$ and $y$. The difference between the two formulas will be substantial when $x \approx y$.

**Exercise 63:**

Find three floating-point numbers a, b and c for which addition is not associative, i.e., for which:

$$(a + b) + c \neq a + (b + c)$$

```
# 63 ------------------------------------------------------------------------
#Find three floating-point numbers a, b and c for which addition is not associative, i.e., fo
#which (a +b)+c   = a+(b+c).

a<- 1
b<- 10^16
c<--10^16

(a+b) + c
```

```
[1] 0
```

```
a + (b+c)
```

```
[1] 1
```

First case: Because you are adding 1 to $and the Mantissa has only roughly 15.9 decimal elements #of precission, the system does not have enough bits to represent the 1 at the end. #therefore it is absorbet and the result is 10^16 # when you now add -10^16 you get 0.

Second case: Since b and c have the same Magnitude, they cancel out =0 #when you then add a you get 1.

Sidenote: If you choose a = 5 and b/c to $10^{16}$ and $-10^{16}$, you can see that the result is 4 and 5. This is because the underlying logic is bit not 10 -> the last bit is the bit which represents the 1 and 0. therefore we can only move in 2 complements. If you increase to ^17 or higher this phenomena will then diminish, because mor bits are out of the storage.

### Exercise 64:

The natural logarithm and exponential functions are inverses of each other, so that mathematically $\log(\exp(x)) = \exp(\log(x)) = x$. Show by example that this property does not hold exactly in computer arithmetic. Does it hold "approximately"?

```
# log(exp(x)) = x fails for x = 0.1
x <- 0.1
log(exp(x)) == x
```

```
[1] FALSE
```

```
log(exp(x)) - x
```

```
[1] 6.938894e-17
```

```
# exp(log(x)) = x also fails for x = 0.1
exp(log(x)) == x
```

```
[1] FALSE
```

```
exp(log(x)) - x
```

```
[1] 1.387779e-17
```

```
# approximately: all.equal allows for a small relative difference
x <- c(0.1, 0.5, 1.5, 2, 3, 5, 10, 100)
all.equal(log(exp(x)), x)
```

```
[1] TRUE
```

```r
all.equal(exp(log(x)), x)
```

```
[1] TRUE
```

```r
# but not for extreme values
log(exp(1000)) #exp(1000) overflows
```

```
[1] Inf
```

```r
log(exp(-1000)) #exp(-1000) underflows to 0
```

```
[1] -Inf
```

```r
log(exp(1e-20)) #exp(1e-20) is rounded to 1
```

```
[1] 0
```

So the property does not hold exactly: $\log(\exp(0.1))$ is off by about $6.9 \cdot 10^{-17}$ and $\exp(\log(0.1))$ by about $1.4 \cdot 10^{-17}$, because both functions return rounded results.

It does hold approximately for moderate values of $x$ (the relative errors above are around $10^{-16}$, and the default tolerance of `all.equal` is around $1.5 \cdot 10^{-8}$, therefore it returns `TRUE`), but not always:

- For large $x$ like 1000, $\exp(x)$ overflows to `Inf` and for large negative $x$ like $-1000$, it underflows to 0, so $\log(\exp(x))$ is `Inf` or `-Inf`.
- For small $x$ like $10^{-20}$, $\exp(x)$ is rounded to exactly 1, so $\log(\exp(x)) = 0$ and all information about $x$ is lost.

**Exercise 65:**

Write a program to compute $e$, the base of natural logarithms, from the definition $e = \lim_{n\to\infty}(1+1/n)^n$. Specifically, compute $(1+1/n)^n$ for $n = 10^k$, $k = 1, \ldots, 20$. Determine the error in your successive approximations by comparing them with the value of `exp(1)`. Does the error always decrease as $n$ increases? Explain your results.

Mathematically $(1+1/n)^n$ gets closer to $e$ when $n$ grows and the error is about $e/(2n)$. On a computer there is a second error: $1+1/n$ is rounded to about 16 digits and raising to the power $n$ multiplies this small rounding error by $n$. So for small $n$ the first error dominates, for large $n$ the rounding error dominates.

```r
e <- function(n) {(1 + 1/n)^n}

k <- 1:20
n <- 10^k
e <- e(n)
print(e, digits = 10)
```

```
 [1] 2.593742460 2.704813829 2.716923932 2.718145927 2.718268237 2.718280469
 [7] 2.718281694 2.718281798 2.718282052 2.718282053 2.718282053 2.718523496
[13] 2.716110034 2.716110034 3.035035207 1.000000000 1.000000000 1.000000000
[19] 1.000000000 1.000000000
```

```r
error <- abs(e - exp(1)) #absolute error
error
```

```
 [1] 1.245394e-01 1.346800e-02 1.357896e-03 1.359016e-04 1.359127e-05
 [6] 1.359363e-06 1.343265e-07 3.011992e-08 2.235374e-07 2.247757e-07
[11] 2.248981e-07 2.416676e-04 2.171794e-03 2.171794e-03 3.167534e-01
[16] 1.718282e+00 1.718282e+00 1.718282e+00 1.718282e+00 1.718282e+00
```

```r
which.min(error) #to find the smallest one
```

```
[1] 8
```

```r
which.max(error) #to find the largest one
```

```
[1] 16
```

```r
#checks
1 + 1/10^16 == 1
```

```
[1] TRUE
```

No, the error doesn't always decrease. It gets smaller until $k = 8$ and then grows again, because the rounding error of `1 + 1/n` is multiplied by about $n$ when we raise it to the power $n$. For example, for the biggest one $k = 16$, when $k \geq 16$ we have `1 + 1/n == 1`, so the result is exactly 1 and the error is $e - 1 \approx 1.718$. So on a computer we can only approximate $e$ with about 8 correct digits in this way.

**Exercise 66:**

```
approx_e <- function(n = NULL, h = NULL) {
  if (!is.null(n)) { # first definition with n
    return((1 + 1/n)^n)
  }
  else if (!is.null(h)) { # second definition with h
    return((1 + h)^(1/h))
  }
  else {
    stop("Please provide either n or h.")
  }
}

approx_e(n = 1e6)
```

```
[1] 2.71828
```

```
approx_e(h = 1e-6)
```

```
[1] 2.71828
```

```
exp(1)
```

```
[1] 2.718282
```

To measure the precision we count the correct decimal digits. If the error is around $10^{-d}$, then about $d$ digits are correct, so we use $\lfloor -\log_{10}(\text{error}) \rfloor$.

```
correct_digits <- function(approx) {
  error <- abs(approx - exp(1))
  if (error == 0) {
    return(16) # matches exp(1) in every digit a double can store
  }
  max(0, floor(-log10(error)))
}
```

**First try: powers of 10.** We use $n = 10^k$ (and $h = 10^{-k}$).

```r
k <- 1:16
digits_n <- sapply(k, function(k) correct_digits(approx_e(n = 10^k)))
digits_h <- sapply(k, function(k) correct_digits(approx_e(h = 10^-k)))
data.frame(k, digits_n, digits_h)
```

```
    k digits_n digits_h
1   1        0        0
2   2        1        1
3   3        2        2
4   4        3        3
5   5        4        4
6   6        5        5
7   7        6        6
8   8        7        7
9   9        6        6
10 10        6        6
11 11        6        6
12 12        3        3
13 13        2        2
14 14        2        2
15 15        0        0
16 16        0        0
```

With powers of 10 we only get about 7 correct digits (best around $n = 10^8$). After that it gets worse again, and from $n = 10^{16}$ on the result is just 1.

The reason is that $1/n = 10^{-k}$ can't be stored exactly in binary, so $1 + 1/n$ is rounded with a relative error of up to $\varepsilon/2 \approx 1.1 \cdot 10^{-16}$. Raising it to the power $n$ multiplies this error by roughly $n$. So the total error has two parts:

$$\text{error} \approx \underbrace{\frac{e}{2n}}_{\text{from the formula}} + \underbrace{e \cdot n \cdot \frac{\varepsilon}{2}}_{\text{from rounding}}.$$

The first part gets smaller as $n$ grows, the second part gets bigger. They are about equal at $n \approx 1/\sqrt{\varepsilon} \approx 10^8$, where the error is about $10^{-8}$, so we get about 7 to 8 digits.

**Better: powers of 2.** If we take $n = 2^k$, then $1/n = 2^{-k}$ is stored exactly, and $1 + 2^{-k}$ is also exact as long as $2^{-k} \geq \varepsilon = 2^{-52}$ (the smallest number with $1 + \varepsilon \neq 1$, see exercise 55). So there is no rounding error in $1 + 1/n$ anymore, and only the error $\frac{e}{2n}$ from the formula is left.

```r
k <- c(10, 20, 30, 40, 50, 51, 52, 53)
digits_n <- sapply(k, function(k) correct_digits(approx_e(n = 2^k)))
digits_h <- sapply(k, function(k) correct_digits(approx_e(h = 2^-k)))
data.frame(k, digits_n, digits_h)
```

```
   k digits_n digits_h
1 10        2        2
2 20        5        5
3 30        8        8
4 40       11       11
5 50       15       15
6 51       15       15
7 52       16       16
8 53        0        0
```

```r
approx_e(n = 2^52) == exp(1)
```

```
[1] TRUE
```

The best choice is $n = 2^{52}$ (or $h = 2^{-52}$): this is the largest $n$ for which $1 + 1/n$ is still exact, so the formula error $\frac{e}{2 \cdot 2^{52}} \approx 3 \cdot 10^{-16}$ is as small as it can get. The result is the same double as `exp(1)`, so we get all 15 to 16 decimal digits that double precision can hold. For $n = 2^{53}$, $1 + 2^{-53}$ rounds to exactly 1, and the result collapses to 1.

So the answer is: with powers of 10 about 7 to 8 digits, with $n = 2^{52}$ the full precision of about 15 to 16 decimal digits.

### Exercise 67:

According to Wilson's Theorem, a fail-safe test of whether a given number $n$ is prime or not is to check if $n$ divides $(n-1)! + 1$. If it does, it is sure to be prime, if not, it is a composite number. Write a function that implements a primality test based on Wilson's Theorem and test all the integers from 2 up to 25 for their primality using your function. Is this a good way to test for primality using a computer? For which number does it start to fail and why?

```r
wilson_test <- function(n) {

  big_number <- factorial(n-1) + 1
  remainder  <- big_number %% n
```

```
  if(remainder == 0) {
    return(TRUE)
  } else {
    return(FALSE)
  }
}

integers <- seq(2,25)
for(i in integers) {
  cat(i, ":", wilson_test(i),"\n")
}
```

```
2 : TRUE
3 : TRUE
4 : FALSE
5 : TRUE
6 : FALSE
7 : TRUE
8 : FALSE
9 : FALSE
10 : FALSE
11 : TRUE
12 : FALSE
13 : TRUE
14 : FALSE
15 : FALSE
16 : FALSE
17 : TRUE
18 : FALSE
19 : TRUE
20 : TRUE
21 : TRUE
22 : TRUE

Warning in wilson_test(i): probable complete loss of accuracy in modulus

23 : FALSE

Warning in wilson_test(i): probable complete loss of accuracy in modulus

24 : TRUE
```

```
Warning in wilson_test(i): probable complete loss of accuracy in modulus
```

```
25 : FALSE
```

We can see that for $n = 20$ we already have an incorrect result. The reason is that the factorial is represented as a "double" in R.

```
typeof(factorial(5))
```

```
[1] "double"
```

This means that after certain $n$, this `factorial(n-1)` is no longer represented exactly. In fact, above $2^{53}$ the gap between consecutive doubles is at least 2. The Wilson's test depends on adding 1 to the factorial, so this $+1$ may be rounded up after certain $n$.

```
# Example for a double's step in R after 2^53
2^53+2 == 2^53
```

```
[1] FALSE
```

```
2^53+1 == 2^53
```

```
[1] TRUE
```

From the output of the function above we can see that the first wrong answer is at $n = 20$. This happens, because $(20-1)! > 2^{53}$. Then we have

```
factorial(20-1) + 1 == factorial(20-1)
```

```
[1] TRUE
```

Then the answer is wrong because 19! is divisible by 20.

Moreover, $(n-1)!$ grows faster than exponentially, which means that its computation is slow and overflows early. Hence, this is not a good approach to test if a number is prime with a computer.

**Exercise 68:**

$$\lim_{x \to 0^+} \frac{e^x - 1}{x} = \frac{1-1}{0} = \frac{0}{0}$$

We can use L'Hôpital's rule:

$$(e^x - 1)' = e^x, \qquad x' = 1$$

$$\lim_{x \to 0^+} \frac{e^x}{1} = \frac{1}{1} = 1$$

**Part (b): The Problem of Catastrophic Cancellation**

When computing $f(x) = \frac{e^x - 1}{x}$ for extremely small values of $x$ (e.g., $10^{-15}$), the term $e^x$ evaluates to a number extremely close to 1.

In finite-precision floating-point systems, computing $e^x - 1$ forces the subtraction of two nearly identical numbers. This triggers catastrophic cancellation (loss of significance): the accurate leading digits cancel each other out, leaving only the "noisy" rounding errors in the least significant bits. When this corrupted, highly inaccurate residual is subsequently divided by the exact, tiny value of x, the relative error explodes. This causes the output to diverge drastically from the theoretical expectation of 1 and eventually collapse to 0.

**Part (c): Numerical Stability through Error Correlation**

The mathematically equivalent formulation $f(x) = \frac{(e^x - 1)}{\ln(e^x)}$ resolves this issue by synchronizing the floating-point inaccuracies. When the computer evaluates $e^x$, it stores a slightly rounded, machine-re-presentable intermediate value—let's call this approximation y. Instead of mixing a rounded intermediate value with an exact input value (as in part b), the new formulation evaluates $\frac{(y-1)}{ln(y)}$ . In this structure, both the numerator and the denominator are subjected to the exact same perturbed input y. Because the natural logarithm mathematically corresponds to this specific y, the approximation errors in the numerator and denominator are perfectly correlated. As long as y does not round to exactly 1 due to machine limits, the division naturally stabilizes the result, allowing the system to reliably return 1.

The problem in this example is the mantissa: In the IEEE 754 standard used by R, the mantissa has a fixed capacity of 53 bits, which provides roughly 15.9 significant decimal digits. When $x = 10^{-15}$, the value of $e^x$is approximately 1.000000000000001. The leading 1 and the long sequence of zeros consume almost all the available space in the mantissa. Any finer, mathematically exact fractional digits that should follow simply do not fit into the memory and are truncated. When the formula computes $e^{x-1}$, that leading 1 is subtracted, leaving the mantissa virtually empty. The only thing remaining is the inaccurate "noise" that happened to be stored in the very last bit.

Ultimately, the mantissa causes the error because it runs out of space to accurately record a microscopically small change alongside a large leading digit. The significant digits wipe each other out, resulting in a total loss of information.

```
#a See Word with L hospital

#b
#(e^x -1) / x

# Vektor für k from 1 to 15
k <- 1:15

# x-Values
x <- 10^(-k)

# Function (b): Standard
f_b <- function(x) {
  (exp(x) - 1) / x
}

# Function (c): Mathematic equivalent
f_c <- function(x) {
  (exp(x) - 1) / log(exp(x))
}

# Outcomes
res_b <- f_b(x)
res_c <- f_c(x)

# Data Frame with the different settings
results <- data.frame(
  k = k,
  x = x,
  Values_B = res_b,
  values_C = res_c
)


options(digits = 10) #sets digits after comma
print(results)
```

```
    k     x     Values_B    values_C
```

```
1    1 1e-01 1.0517091808 1.051709181
2    2 1e-02 1.0050167084 1.005016708
3    3 1e-03 1.0005001667 1.000500167
4    4 1e-04 1.0000500017 1.000050002
5    5 1e-05 1.0000050000 1.000005000
6    6 1e-06 1.0000005000 1.000000500
7    7 1e-07 1.0000000494 1.000000050
8    8 1e-08 0.9999999939 1.000000005
9    9 1e-09 1.0000000827 1.000000001
10 10 1e-10 1.0000000827 1.000000000
11 11 1e-11 1.0000000827 1.000000000
12 12 1e-12 1.0000889006 1.000000000
13 13 1e-13 0.9992007222 1.000000000
14 14 1e-14 0.9992007222 1.000000000
15 15 1e-15 1.1102230246 1.000000000
```

```
#We see:
# For b at first it gets closer to 1, as we expect.
#but at k around 8 it starts to fall out of line with our theory.
```

**Exercise 69:**

Write a program to compute an approximate value for the derivative of a function using the finite-difference formula

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}.$$

Test your program using the function $\tan(x)$ for $x = 1$ (remember that $\tan'(x) = 1 + \tan(x)^2$).

```
# f is a function, x the point and h the step size
finite_diff <- function(f, x, h) {
  return((f(x + h) - f(x)) / h)
}

x <- 1
exact <- 1 + tan(x)^2
print(exact, digits = 16)
```

```
[1] 3.42551882081476
```

```r
# example: h = 1e-5
finite_diff(tan, x, 1e-5)
```

```
[1] 3.425572171
```

```r
# try h = 10^(-1), ..., 10^(-16)
h <- 10^(-(1:16))
approx <- finite_diff(tan, x, h)
err <- abs(approx - exact)

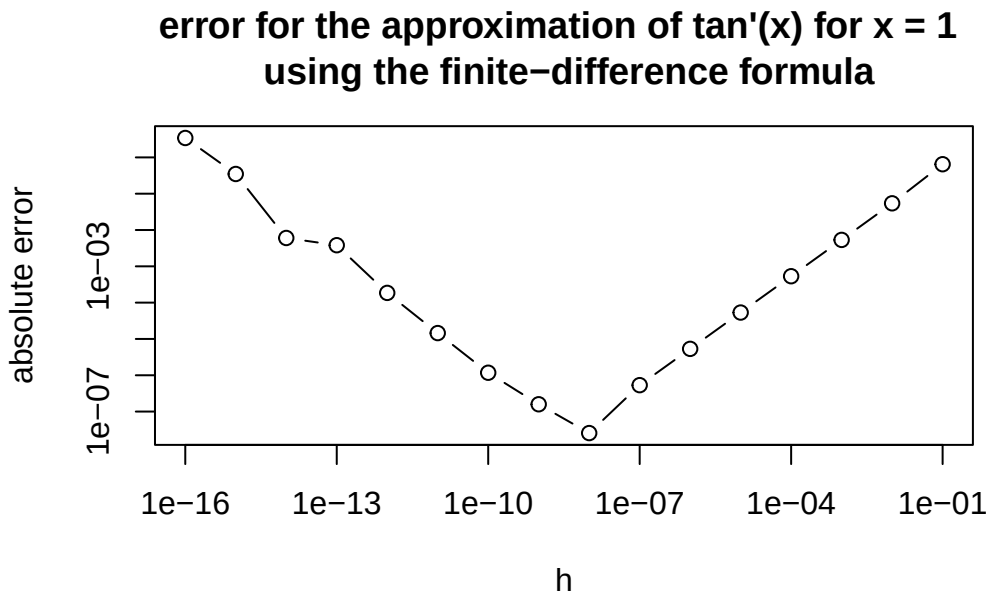
# h with the smallest error
h[which.min(err)]
```

```
[1] 1e-08
```

```r
min(err)
```

```
[1] 2.554135259e-08
```

```r
#plotting the errors
plot(h, err, log = "xy", type = "b", xlab = "h", ylab = "absolute error", main =
"error for the approximation of tan'(x) for x = 1 \n using the finite-difference formula")
```



The exact value is $1 + \tan(1)^2 = 3.425518820814759$. The error first decreases with $h$ and is smallest for $h = 10^{-8}$ (error about $2.6 \cdot 10^{-8}$), and then it increases again. This happens because there are two errors at the same time:

- The truncation error shrinks as $h$ gets smaller.
- The rounding error grows as $h$ gets smaller, since $f(x + h) - f(x)$ is a subtraction of two almost equal numbers. For $h = 10^{-16}$, $1 + h$ is exactly 1, so $f(x + h) = f(x)$, the approximation is 0 and the error equals the derivative itself.

**Exercise 70:**

As every schoolchild learns, the two solutions of the quadratic equation $ax^2 + bx + c = 0$ are given by the formula

$$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

For $|b| \approx \sqrt{b^2 - 4ac}$ it is better to compute one of the roots using the alternative formula

$$\frac{2c}{-b \mp \sqrt{b^2 - 4ac}}$$

Why?

If we have the case, that $|b| \approx \sqrt{b^2 - 4ac}$, then $4ac$ is very small compared to $b^2$. In the school formula one of the roots then subtracts two almost equal numbers, for $b > 0$ this is $-b + \sqrt{b^2 - 4ac}$. The result is close to 0 and most of the significant digits cancel, so the rounding errors of the square root become a significant part of the result.

The alternative formula comes from $x_1 \cdot x_2 = c/a$ (Vieta). The other root $x_2 = \frac{-b-\sqrt{b^2-4ac}}{2a}$ adds two numbers of the same sign, so it is accurate and then

$$x_1 = \frac{c}{a\, x_2} = \frac{2c}{-b - \sqrt{b^2 - 4ac}}.$$

Here the denominator is a sum of two numbers with the same sign, so there is no cancellation.

We can compare 2 formulas. Let's take $a = 1$, $b = 10^8$, $c = 1$ because the problem appears only when $b^2$ is much bigger than $4ac$. Then $\sqrt{b^2 - 4ac}$ is almost the same number as $b$ and in $-b + \sqrt{b^2 - 4ac}$ we subtract two almost equal numbers. With small numbers like $b = 10$ the error is about $10^{-15}$, so we cannot see it. With $b = 10^8$ the error is big enough to see, which we show now.

The correct value of the small root is $x \approx -c/b = -10^{-8}$, because $x$ is so small that $ax^2$ is almost 0 and the equation $ax^2 + bx + c = 0$ becomes $bx + c \approx 0$.

```r
#classical school formula
classic_s<- function(a, b, c) {(-b + sqrt(b^2 - 4 * a * c)) / (2 * a)}

#alternative formula with same root
alter <- function(a, b, c) { 2 * c / (-b - sqrt(b^2 - 4 * a * c))}

classic_s(1, 10^8, 1)
```

```
[1] -7.450580597e-09
```

```r
alter(1, 10^8, 1)
```

```
[1] -1e-08
```

```r
abs(classic_s(1, 10^8, 1) + 1e-8) / 1e-8 #compare to the root
```

```
[1] 0.2549419403
```

```r
abs(alter(1, 10^8, 1) + 1e-8) / 1e-8
```

```
[1] 0
```

So, the school formula subtracts two almost equal numbers for one of the roots, which loses many significant digits, like 25% in this example. In the second formula the numbers in the denominator are added and have the same sign, so there is no cancellation and we checked relative error is about 0.

**Exercise 71:**

(a) For $x$ close to 0, both $\frac{1}{1-x}$ and $\frac{1}{1+x}$ are almost 1. Subtracting two nearly equal numbers cancels the leading digits (catastrophic cancellation), so the small rounding errors in the two terms become large relative to the result.

```r
g <- function(x) {
  1 / (1 - x) - 1 / (1 + x)
}

# for small x the result should be about 2x
x_values <- c(1e-1, 1e-5, 1e-9, 1e-12, 1e-15)
data.frame(x = x_values, g = g(x_values), should_be = 2 * x_values)
```

```
      x                g should_be
1 1e-01 2.020202020e-01     2e-01
2 1e-05 2.000000000e-05     2e-05
3 1e-09 2.000000165e-09     2e-09
4 1e-12 2.000177801e-12     2e-12
5 1e-15 2.220446049e-15     2e-15
```

For $x = 10^{-12}$ only about 4 digits are correct, and for $x = 10^{-15}$ the result is about 10% off. At $x = \pm 1$ the expression is not defined at all, since we divide by zero.

(b) Putting both fractions over a common denominator gives

$$\frac{1}{1-x} - \frac{1}{1+x} = \frac{(1+x)-(1-x)}{(1-x)(1+x)} = \frac{2x}{1-x^2}.$$

There is no subtraction of nearly equal numbers anymore, so this works well for small $x$:

```r
g_better <- function(x) {
  (2 * x) / (1 - x^2)
}

data.frame(x = x_values, g_better = g_better(x_values), should_be = 2 * x_values)
```

```
      x       g_better should_be
1 1e-01 2.02020202e-01     2e-01
2 1e-05 2.00000000e-05     2e-05
3 1e-09 2.00000000e-09     2e-09
4 1e-12 2.00000000e-12     2e-12
5 1e-15 2.00000000e-15     2e-15
```

**Further improvement.** $\frac{2x}{1-x^2}$ still has a problem when $x$ is close to $\pm 1$: then $x^2$ is rounded first, and $1 - x^2$ is again a subtraction of two nearly equal numbers, so that rounding error gets blown up. If we keep the denominator factored,

$$\frac{2x}{(1-x)(1+x)},$$

this does not happen. For $x$ close to 1, $1 - x$ is computed exactly in floating point (both numbers are within a factor 2 of each other), and $1 + x$ has no cancellation at all. Near $-1$ it is the same with the roles swapped.

```r
g_best <- function(x) {
  (2 * x) / ((1 - x) * (1 + x))
}

# close to 1 the two rearrangements start to differ
x_near_1 <- c(1 - 1e-5, 1 - 1e-7, 1 - 1e-9)
rel_diff <- abs(g_better(x_near_1) - g_best(x_near_1)) / abs(g_best(x_near_1))
data.frame(one_minus_x = 1 - x_near_1, g_better = g_better(x_near_1),
           g_best = g_best(x_near_1), rel_diff = rel_diff)
```

```
      one_minus_x       g_better         g_best        rel_diff
1 1.000000000e-05      99999.500      99999.500 4.137130185e-13
2 9.999999995e-08    9999999.505    9999999.505 3.996379871e-11
3 9.999999717e-10 1000000027.282 1000000027.782 5.000001053e-10
```

```r
# and for small x it is just as good as g_better
data.frame(x = x_values, g_best = g_best(x_values), should_be = 2 * x_values)
```

```
      x         g_best should_be
1 1e-01 2.02020202e-01     2e-01
2 1e-05 2.00000000e-05     2e-05
3 1e-09 2.00000000e-09     2e-09
4 1e-12 2.00000000e-12     2e-12
5 1e-15 2.00000000e-15     2e-15
```

The relative difference grows as $x$ gets closer to 1 (up to about $5 \cdot 10^{-10}$ here), and that error comes from `g_better`, because there $x^2$ is rounded before the subtraction. So $\frac{2x}{(1-x)(1+x)}$ is accurate both near 0 and near $\pm 1$. At exactly $x = \pm 1$ the expression is still undefined, but that's a mathematical limit, not a numerical one.